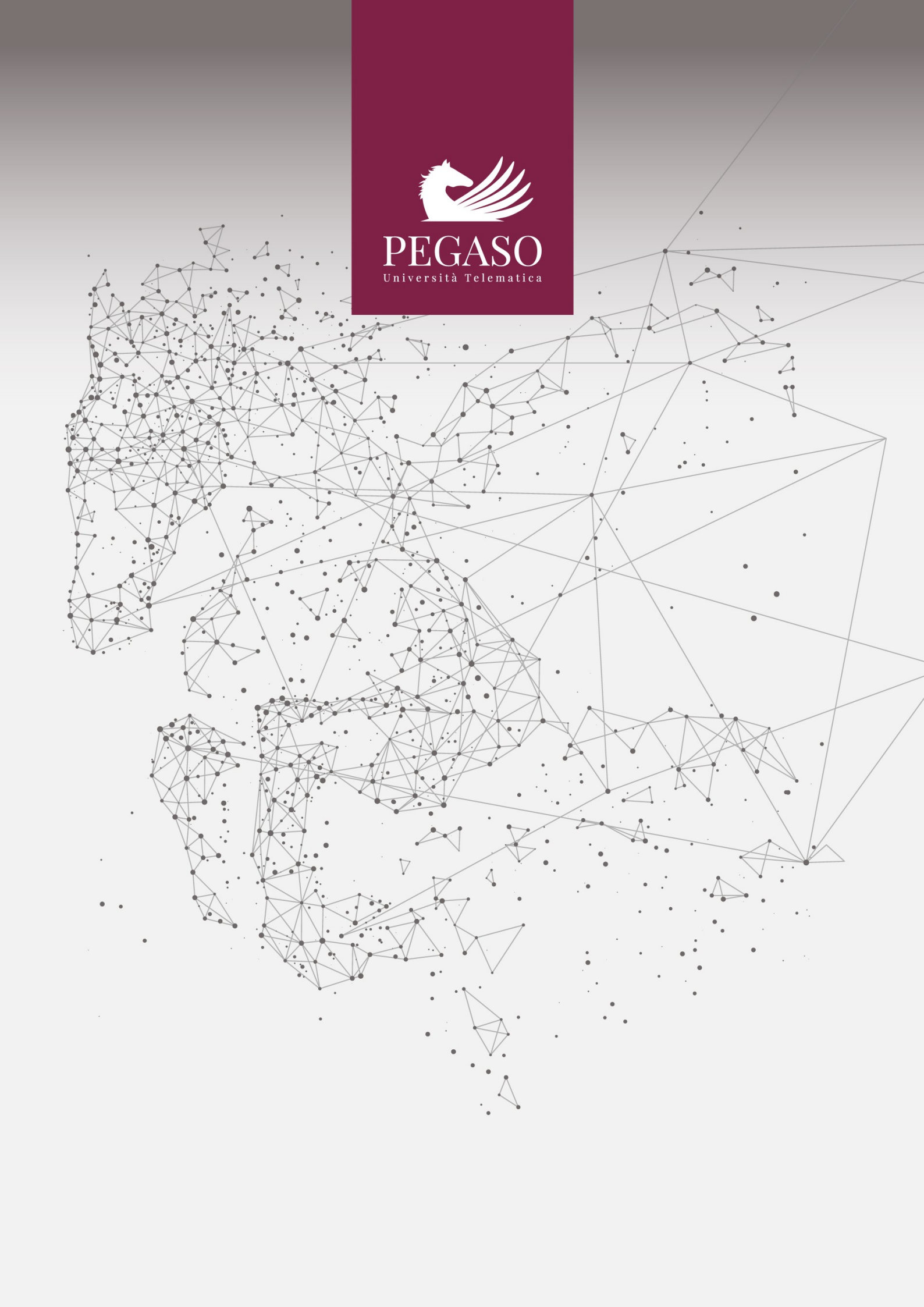

PEGASO
Università Telematica

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

# Indice

**1. SOAP E REST ..... 3**

**2. JSON ..... 4**

**3. XML VS JSON ..... 10**

**BIBLIOGRAFIA E SITOGRAFIA ..... 11**

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

# 1. SOAP e REST

**OAP** (**Simple Object Access Protocol**) è un protocollo utilizzato per la comunicazione tra sistemi in un'architettura **SOA** (Service-Oriented Architecture). Le API progettate con **SOAP** utilizzano **XML** per il formato dei messaggi e ricevono le richieste tramite HTTP o SMTP. SOAP semplifica la condivisione delle informazioni per le app in esecuzione in ambienti diversi o scritte in lingue diverse.

Le API Web che aderiscono ai vincoli dell'architettura **REpresentational State Transfer** (**REST**) sono denominate **RESTful API** e sono prevalentemente usate per creare **microservices**. REST differisce da SOAP in modo fondamentale: SOAP è un protocollo, mentre REST è uno stile architetturale: ciò significa che non esiste uno standard ufficiale per le web RESTful API, tuttavia tali API si basano sull'utilizzo di standard **HTTP** (Hypertext Transfer Protocol) come protocollo di comunicazione tra client e server, ed utilizza le principali verbi HTTP (GET, POST, PUT, DELETE) per effettuare operazioni **CRUD** (Create, Read, Update, Delete) sulle risorse identificate da **URI** (Uniform Resource Identifier). Inoltre, REST prevede l'uso di formati standard come **JSON** o **XML** per la rappresentazione delle risorse.

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

# 2. JSON

**JSON** (**JavaScript Object Notation**) è un semplice formato per lo scambio di dati. Per le persone è facile da leggere e scrivere, mentre per le macchine risulta facile da generare e analizzarne la sintassi. Si basa su un sottoinsieme del Linguaggio di Programmazione JavaScript, Standard ECMA-262 Terza Edizione - Dicembre 1999.

JSON è un formato di testo completamente indipendente dal linguaggio di programmazione, ma utilizza convenzioni conosciute dai programmatori di linguaggi della famiglia del C, come C, C++, C#, Java, JavaScript, Perl, Python, e molti altri. Questa caratteristica fa di JSON un linguaggio ideale per lo scambio di dati.

JSON è basato su due strutture:

- Un insieme di coppie nome/valore. In diversi linguaggi, questo è realizzato come un oggetto, un record, uno struct, un dizionario, una tabella hash, un elenco di chiavi o un array associativo.
- Un elenco ordinato di valori. Nella maggior parte dei linguaggi questo si realizza con un array, un vettore, un elenco o una sequenza.

Queste sono strutture di dati universali. Virtualmente tutti i linguaggi di programmazione moderni li supportano in entrambe le forme. È sensato che un formato di dati che è interscambiabile con linguaggi di programmazione debba essere basato su queste strutture.

In JSON:

- Un oggetto è una serie non ordinata di nomi/valori.
- Un oggetto inizia con { (parentesi graffa sinistra).
- Un oggetto finisce con } (parentesi graffa destra).
- Ogni nome è seguito da : (due punti).
- Le coppie di nome/valore sono separata da , (virgola).

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

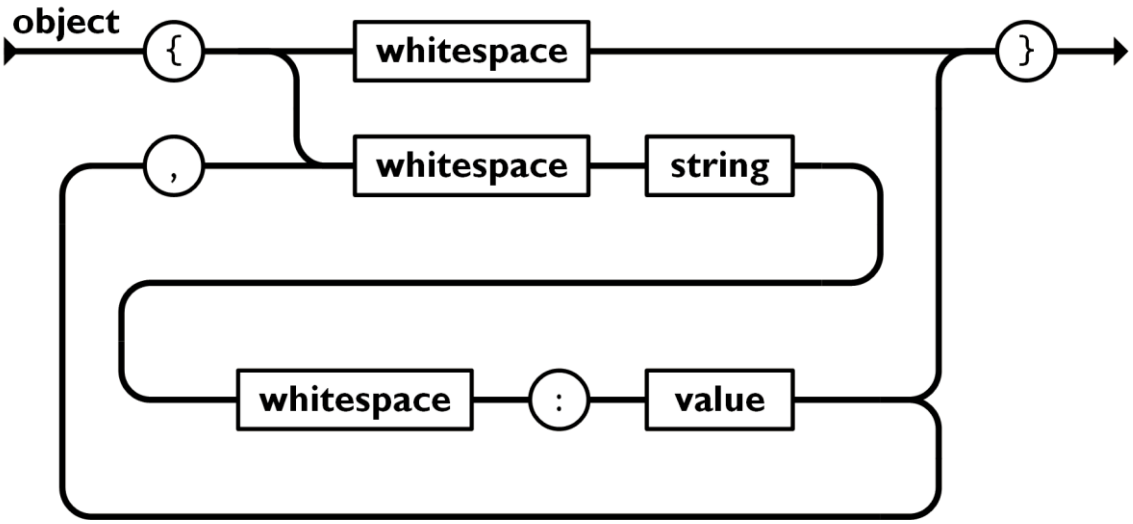



***Figura 1** - JSON object*

Un array è una raccolta ordinata di valori:

- Un array comincia con [ (parentesi quadra sinistra).
- Un array finisce con ] (parentesi quadra destra).
- I valori sono separati da , (virgola).

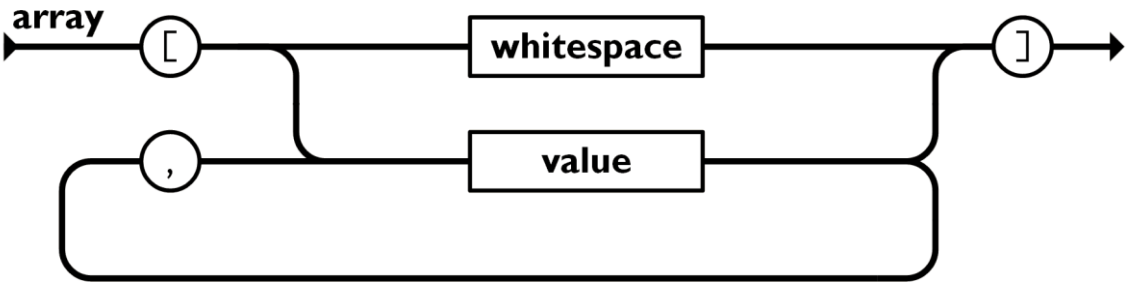



***Figura 2** - JSON Array*

Un valore può essere una stringa tra virgolette, o un numero, o vero true o falso false o nullo null, o un oggetto o un array. Queste strutture possono essere annidate.

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

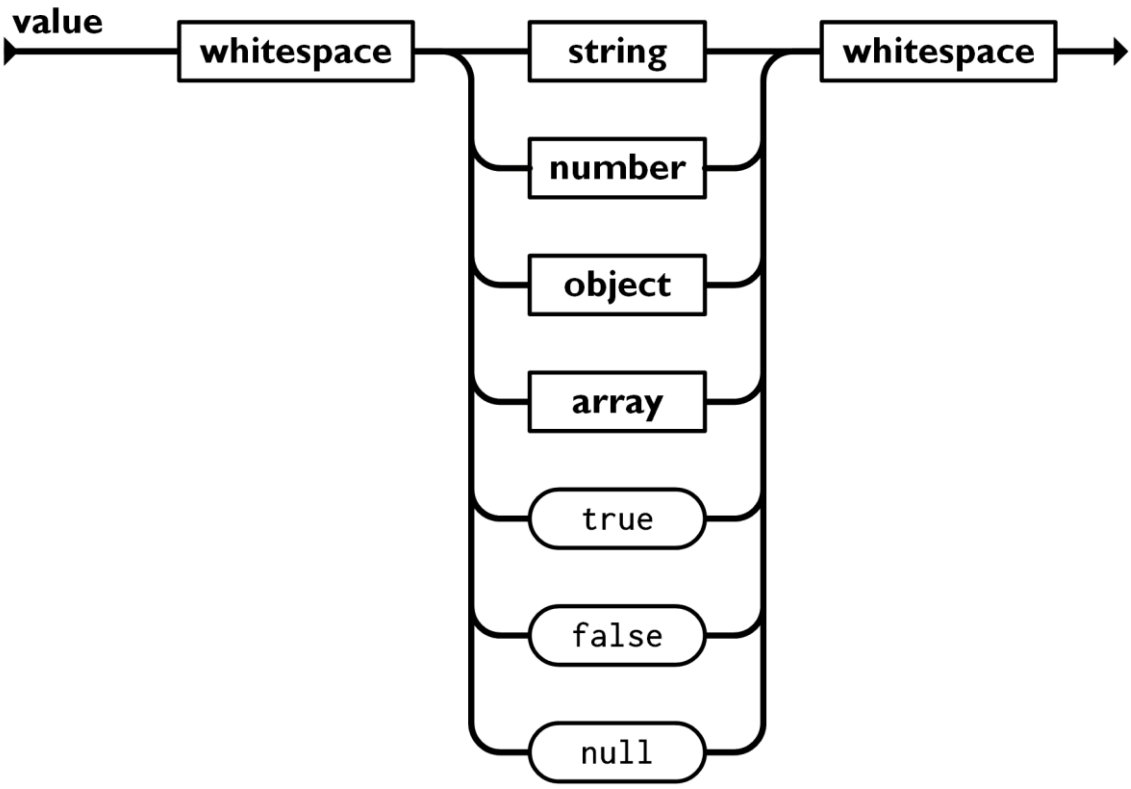



***Figura 3** - JSON valore*

Una stringa è una raccolta di zero o più caratteri Unicode, tra virgolette; per le sequenze di escape utilizza la barra rovesciata. Un singolo carattere è rappresentato come una stringa di caratteri di lunghezza uno. Una stringa è molto simile ad una stringa C o Java.

Un numero è molto simile ad un numero C o Java, a parte il fatto che i formati ottali e esadecimali non sono utilizzati.

I caratteri di spaziatura possono essere inseriti in mezzo a qualsiasi coppia di token.

A parte alcuni dettagli di codifica, questo descrive totalmente il linguaggio.

Vediamo dunque alcuni esempi "pratici":

- nome/valore: *"name":"Mario"*
- lista di nome/valore: *"name":"Mario", "surname":"Rossi"*
- oggetto con 1 proprietà (nome): *{"name":"Mario"}*
- array: *"cars": ["Ferrari", "Red Bull", "McLaren"]*
- oggetto con 2 proprietà (nome e cognome): *{"name":"Mario","cognome":"Rossi}*

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

Riportiamo di seguito un esempio di XML, del corrispondente JSON e della sua versione in "albero":

XML

```
<formula1>
    <teams>
        <team>
            <name>Ferrari</name>
            <nationality>Italy</nationality>
            <drivers>
                <driver>Leclers</driver>
                <driver>Sainz</driver>
            </drivers>
        </team>
        <team>
            <name>Red Bull</name>
            <nationality>Austria</nationality>
            <drivers>
                <driver>Verstappen</driver>
                <driver>Perez</driver>
            </drivers>
        </team>
    </teams>
</formula1>
```

JSON

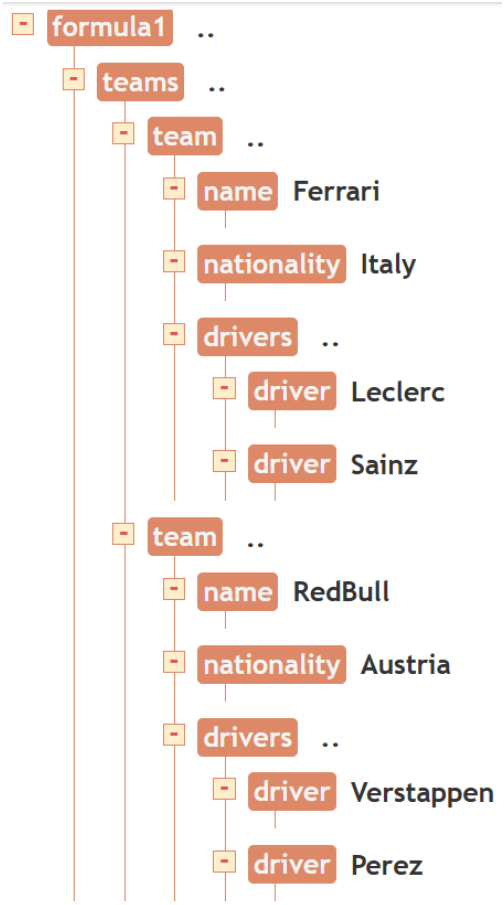
```
{
  "formula1": {
    "teams": {
      "team": [
        {
          "name": "Ferrari",
          "nationality": "Italy",
          "drivers": {
            "driver": [
              "Leclers",
              "Sainz"
            ]
          }
```

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

```
            },
            {
              "name": "Red Bull",
              "nationality": "Austria",
              "drivers": {
                "driver": [
                  "Verstappen",
                  "Perez"
                ]
              }
            }
          ]
        }
      }
    }
```

ALBERO




PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

Tale visualizzazione è stata ottenuta tramite CodeBeauty: Best Online XML Viewer, XML Formatter, XML Editor, Analyser, Beautify-Beautifier, Minify, Tree structure (codebeautify.org)

Il formato JSON è sintatticamente simile al codice per la creazione di oggetti JavaScript. Per questo motivo, un programma JavaScript può facilmente convertire i dati JSON in oggetti JavaScript.

JavaScript ha una funzione incorporata per convertire le stringhe JSON in oggetti JavaScript:

```
JSON.parse()
```

JavaScript ha anche una funzione per convertire un oggetto in una JSON string:

```
JSON.stringify()
```

Se ad es. ricevessimo da un web server il seguente testo:

```
'{"nome":"Mario", "cognome":"Rossi", "età":30}'
```

Tramite la funzione "parse" possiamo ottenere un oggetto JavaScript:

```
const obj = JSON.parse('{"nome":"Mario", "cognome":"Rossi",
"età":30}');
```

Per accedere al "nome" basterebbe scrivere:

```
obj.nome
```

Qualora si ricevesse un array in risposta, come ad es.

```
'["uno","due","tre","quattro"]'
```

la funzione parse restituirebbe un Javascript Array

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

# 3. XML vs JSON

JSON è come XML perché:

- Sia JSON che XML sono "autodescrittivi" (leggibili dall'uomo).
- Sia JSON che XML sono gerarchici (valori all'interno di valori).
- Sia JSON che XML possono essere analizzati e utilizzati da molti linguaggi di programmazione.
- Sia JSON che XML possono essere recuperati con XMLHttpRequest.

JSON è diverso da XML perché:

- JSON non utilizza il tag di fine.
- JSON è più breve.
- JSON è più veloce da leggere e scrivere.
- JSON può utilizzare gli array.

La più grande differenza è:

- XML deve essere analizzato con un parser XML.
- JSON può essere analizzato da una funzione JavaScript standard.
- Perché JSON è migliore di XML.
- XML è molto più difficile da analizzare rispetto a JSON.
- JSON viene analizzato in un oggetto JavaScript pronto all'uso.

Per le applicazioni AJAX, JSON è più veloce e più semplice di XML, infatti se si usasse XML gli steps sarebbero i seguenti:

- Recupera un documento XML.
- Usa il DOM XML per scorrere il documento.
- Estrai i valori e memorizzali nelle variabili.

Usando JSON invece:

- Recupera una stringa JSON
- JSON. Analizza la stringa JSON

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

# Bibliografia e sitografia


- V. Roberto, M. Frailis, A. Gugliotta, P. Omero. Introduzione alle Tecnologie Web, McGraw-Hill, 2010
- A. S. Tanembaum. Computer Networks (4 ed.). Prentice Hall (4 ed.), 2003.
- https://www.json.org/json-it.html


PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*